

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: INTERNET PROGRAMMING**

**COURSE CODE: CSA4305**

### **COURSE OUTCOME COVERED**

**CO5:** Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design

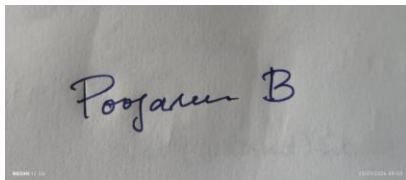
Weightage: 20%

**QUESTIONS:**

**Total Marks: 25**

### ***Code of Conduct:***

*I POOJASREE B (192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

A photograph of a piece of paper with a handwritten signature in blue ink. The signature reads "Poojashree B".

***Signature of the student:***

**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>            | <b>Weight age (%)</b> | <b>Level 4 (Exemplary)</b>                                                        | <b>Level 3 (Proficient)</b>                            | <b>Level 2 (Developing)</b>                    | <b>Level 1 (Beginning)</b>                        |
|----------------------------|-----------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Problem Understanding      | 20%                   | Clearly defines problem, objectives, and all requirements accurately              | Understands problem with minor gaps                    | Partial understanding; misses key requirements | Misinterprets or fails to understand problem      |
| Generation of Solutions    | 25%                   | Develops multiple innovative and feasible alternatives with clear differentiation | Generates relevant alternatives with minor limitations | Limited or repetitive alternatives             | Fails to generate meaningful alternatives         |
| Evaluation of Alternatives | 25%                   | Critically compares alternatives using appropriate criteria and metrics           | Compares alternatives with some justification          | Basic or superficial comparison                | No proper comparison conducted                    |
| Solution Selection         | 20%                   | Selects best solution with strong justification and decision rationale            | Selects suitable solution with moderate justification  | Selection is weakly justified                  | No clear or justified selection                   |
| Feasibility                | 10%                   | Applies relevant concepts ensuring high feasibility and practicality              | Applies concepts with minor issues                     | Limited feasibility or incorrect application   | Solution is impractical or conceptually incorrect |

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.
4. An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5. A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

## 1. REST vs SOAP – Selection of Web Service

### Analysis

| Feature             | REST                   | SOAP                  |
|---------------------|------------------------|-----------------------|
| Architecture        | Resource-based         | Protocol-based        |
| Data format         | Mainly JSON            | Mainly XML            |
| Performance         | Faster and lightweight | Relatively slower     |
| Scalability         | Highly scalable        | More complex to scale |
| Security            | HTTPS, OAuth, JWT      | WS-Security, HTTPS    |
| Mobile applications | Very suitable          | Less suitable         |
| Complexity          | Simple                 | More complex          |

### Selected Approach: REST

For a mobile and web application, **REST API** is the better choice because it is lightweight, fast, and easy to consume from different platforms.

### Proposed Architecture

**Mobile/Web Client → REST API → Application Server → Database**

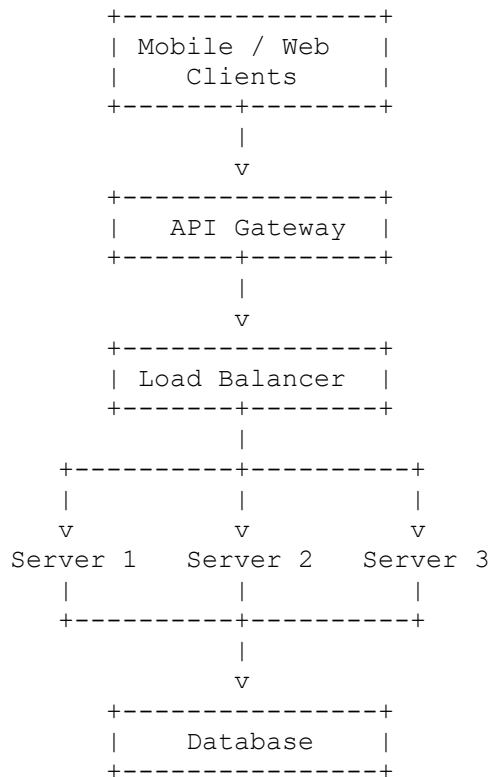
### Justification

- **Performance:** JSON messages are smaller and faster than SOAP XML messages.
- **Scalability:** REST is stateless, allowing requests to be distributed across multiple servers.
- **Security:** HTTPS, OAuth 2.0, JWT, input validation, and API gateways can secure the API.
- **Compatibility:** REST works easily with Android, iOS, browsers, and other clients.

**Conclusion:** REST is preferred for this application because it provides better performance, scalability, and simplicity.

## 2. Scalable Service Architecture

### Basic Architecture



### Working

1. Clients send requests to the API Gateway.
2. The load balancer distributes requests among application servers.
3. Application servers process requests.
4. Servers access the database to read or update data.
5. The response is returned to the client.

### Alternative Design

Use **microservices with caching and asynchronous processing**:

Client



API Gateway



Microservices



Message Queue → Background Workers



Database + Cache

Different functions such as authentication, payment, orders, and notifications can run as separate services.

### **Justification**

- Load balancing prevents one server from becoming overloaded.
- Multiple application servers provide **horizontal scalability**.
- Caching reduces repeated database requests.
- Message queues handle large numbers of background requests.
- Microservices allow individual services to scale independently.
- Database replication improves availability and performance.

### **Conclusion:**

A load-balanced, microservice-based architecture with caching and asynchronous processing provides better scalability for a high-traffic platform.

### 3. Secure Communication Protocol for Banking Application

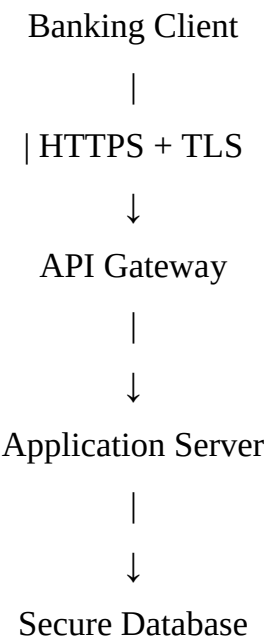
#### Protocol Analysis

| Protocol  | Security                | Reliability | Performance | Suitability             |
|-----------|-------------------------|-------------|-------------|-------------------------|
| HTTP      | Low                     | Good        | High        | ✗                       |
| HTTPS     | High                    | High        | High        | ✓                       |
| FTP       | Low/Medium              | High        | Medium      | ✗                       |
| WebSocket | Can be secured with TLS | High        | Very High   | Conditional             |
| SSH       | Very High               | High        | Medium      | Not for normal web APIs |

#### Selected Protocol: HTTPS

**HTTPS (HTTP over TLS)** is the most suitable protocol for banking applications.

#### Architecture



#### Security Mechanisms

- **TLS encryption** protects data during transmission.
- **Digital certificates** authenticate the server.
- **Authentication** verifies users.

- **Authorization** ensures users can access only permitted resources.
- **JWT/OAuth 2.0** can be used for secure API access.
- Sensitive data should be encrypted at rest.

### **Justification**

HTTPS provides confidentiality, authentication, and integrity while maintaining good performance. It protects sensitive information such as account numbers, passwords, and transaction details from interception and modification.

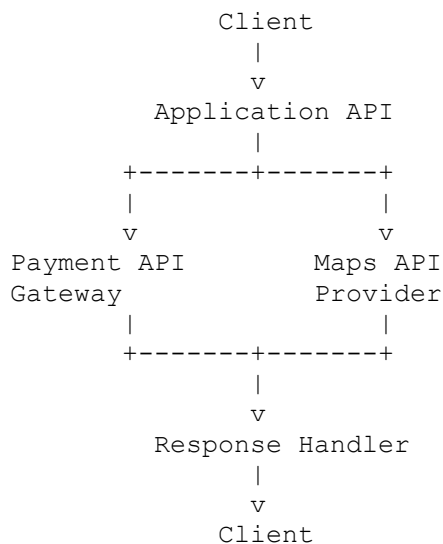
**Conclusion:** HTTPS with TLS is the standard and most appropriate choice for secure banking communication.



## 4. API Integration Strategy for Third-Party Services

Third-party services such as payment gateways, maps, SMS, and email can be integrated through APIs.

### Architecture



### API Integration Process

#### 1. Request Creation

- Application creates a request with required parameters.
- Example: payment amount, currency, and transaction ID.

#### 2. Authentication

- Use API keys, OAuth 2.0, JWT, or signed requests.
- Store credentials securely on the server.

#### 3. Request Handling

- Send the request using HTTPS.
- Set appropriate headers, timeout, and request IDs.

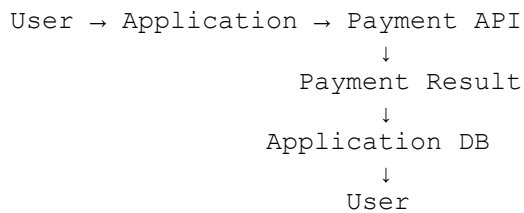
#### 4. Response Processing

- Validate the response.
- Check status codes and response data.
- Update the application database.

## 5. Error Handling

- Handle timeout, invalid requests, authentication failures, and service unavailability.

### Example



### Alternative Approach

Use an **API Gateway or integration service** between the application and third-party providers.

Benefits:

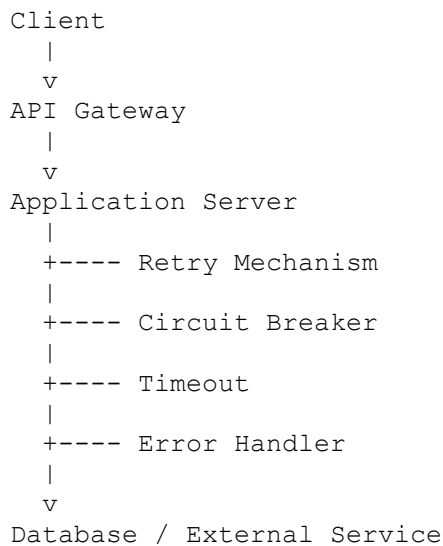
- Centralized authentication.
- Request monitoring and logging.
- Rate limiting.
- Caching where appropriate.
- Retry and failover mechanisms.
- Easier management of multiple external APIs.

**Conclusion:** A centralized API integration layer improves security, maintainability, reliability, and monitoring.

## 5. Error Handling and Fault Tolerance Mechanism

A web service should continue operating even when temporary failures occur.

### Proposed Architecture



### Error Handling Mechanisms

#### 1. Input Validation

- Validate client data before processing.

#### 2. HTTP Status Codes

- 400 – Bad Request
- 401 – Unauthorized
- 403 – Forbidden
- 404 – Not Found
- 500 – Internal Server Error
- 503 – Service Unavailable

#### 3. Retry Mechanism

- Retry temporary failures using controlled delays.
- Exponential backoff can prevent excessive requests.

#### 4. Timeout

- Stop waiting when a service takes too long to respond.

## 5. **Circuit Breaker**

- Temporarily stops requests to a failing service and prevents cascading failures.

## 6. **Logging and Monitoring**

- Record errors, request IDs, timestamps, and service status.

## **Alternative Strategies**

**Caching:** Serve frequently requested data from cache when the main service is temporarily unavailable.

**Message Queue:** Store requests in a queue and process them later if the target service is temporarily unavailable.

**Redundant Servers:** Use multiple servers so traffic can be redirected when one server fails.

## **User Experience**

Instead of displaying technical errors, the application should provide meaningful messages such as:

"The service is temporarily unavailable. Please try again shortly."

## **Conclusion**

Combining **validation, retries, timeouts, circuit breakers, logging, caching, and redundant servers** provides fault tolerance and improves service availability and user experience.